BASIC BACKTRACKING

```
// Try a choice, recurse, undo the choice (backtrack), try the next one.
// Used for generating subsets, permutations, or valid arrangements.
```

General template

```
void backtrack(/* state */) {
    if (/* base case: solution complete */) {
        // record/process the solution
        return;
    }
    for (/* each possible choice */) {
        // make choice
        backtrack(/* updated state */);
        // undo choice (backtrack)
    }
}
```

Demo 1: Generate all subsets of an array

```
vector<int> cur; vector<vector<int>> allSubsets;
void gen(int idx, vector<int>& a) {
    if (idx == a.size()) { allSubsets.push_back(cur); return; }
    gen(idx + 1, a);  // choice 1: don't include a[idx]
    cur.push_back(a[idx]);
    gen(idx + 1, a);  // choice 2: include a[idx]
    cur.pop_back();  // undo (backtrack)
}
```

Demo 2: Generate all permutations of an array

```
void permute(vector<int>& a, int idx) {
    if (idx == a.size()) { /* process a[] as one full permutation */ return; }
    for (int i = idx; i < a.size(); i++) {
        swap(a[idx], a[i]);
        permute(a, idx + 1);
        swap(a[idx], a[i]);  // undo (backtrack)
    }
}
// Alternative: use next_permutation in a loop instead of writing this by hand
sort(a.begin(), a.end());
do { /* use a */ } while (next_permutation(a.begin(), a.end()));
```

Demo 3: Combination Sum (pick numbers that sum to target, reuse allowed)

```
vector<int> cur; vector<vector<int>> results;
void combSum(vector<int>& a, int idx, int remain) {
    if (remain == 0) { results.push_back(cur); return; }
    if (remain < 0 || idx == a.size()) return;
    cur.push_back(a[idx]);
    combSum(a, idx, remain - a[idx]);   // reuse same element
    cur.pop_back();                      // undo (backtrack)
    combSum(a, idx + 1, remain);        // move to next element
}
```